Lecture 10 · Part II · Lecture notes

PyTorch

Dataset: Fashion-MNIST

How to use these notes

The primary text for this lecture is Géron, Chapter 10. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

---

1 Where we are

Lecture 9 found four things `MLPClassifier` will not let you do: change what it optimises, see a gradient, stop mid-epoch, or move the computation to another device. All four are one wall — `fit()` contains a loop over epochs and batches that you did not write, cannot see and cannot enter.

The repair is not a better library. It is *owning the loop*. Today: what that loop has to compute, why it can be computed at all, and how to write it.

2 Derivation: backpropagation as reverse-mode autodiff

You called `clf.fit()`. It computed 266,610 partial derivatives, on every batch, twenty times over. *How?* The cost of that computation is what decides whether training a network is possible at all — and the reason it is possible is a property of the *direction* in which the chain rule is applied.

Method	**Cost for P parameters**	**Exact?**
By hand	your afternoon, per architecture	yes, if you are careful
Finite differences	$P + 1$ evaluations	no — truncation and cancellation
Symbolic	expression blow-up	yes
Automatic differentiation	see below	to machine precision

Automatic differentiation is neither symbolic nor numerical. It differentiates the *program*, operation by operation, using the chain rule and nothing else.

2.1 Two bracketings, two algorithms

A network is a composition, so writing $L = f_3(f_2(f_1(\boldsymbol{\theta})))$ gives

$$\frac{\partial L}{\partial \boldsymbol{\theta}} = J_3 J_2 J_1,$$

with J_i the Jacobian of f_i evaluated at the point the forward pass reached. Every layer contributes one factor — and matrix multiplication is associative:

$$\underbrace{(J_3 J_2) J_1}_{\text{reverse mode}} \;=\; \underbrace{J_3 (J_2 J_1)}_{\text{forward mode}} .$$

Same number. Wildly different cost.

Forward mode picks a direction $\mathbf{v}$ in input space and carries a tangent $\dot{u} = \partial u/\partial v$ alongside every intermediate value. One sweep gives the derivative of every output with respect to *one* input direction, and nothing needs to be stored, because the tangent travels with the value.

Reverse mode picks a direction in output space — for a scalar loss there is only one, and it is 1 — and carries an adjoint $\bar{u} = \partial L/\partial u$ backwards. One sweep gives the derivative of *one* output with respect to every input, and the forward values must be kept, because the Jacobians are evaluated at them.

Mode	Sweeps needed	Cheap when
Forward	n — one per input	few inputs, many outputs
Reverse	m — one per output	many inputs, few outputs

Each sweep costs a small constant times one forward evaluation — around 2 to 4, and it does not depend on n or m. That constant is why the backward pass costs about what the forward pass costs, no matter how many parameters there are.

2.2 A worked example small enough to check

$$L = w^2, \qquad w = xy + \sin x, \qquad \text{at } x = 2,\ y = 3.$$

By hand, $\partial L/\partial x = 2w(y + \cos x)$ and $\partial L/\partial y = 2wx$.

Node	Expression	Value	Adjoint	Computed as	Value
x	input	2.0	$\partial L/\partial L$	the seed	1
y	input	3.0	$\partial L/\partial w$	$2w$	13.8186
u	$x \cdot y$	6.0000	$\partial L/\partial u$	$(\partial L/\partial w) \cdot 1$	13.8186
s	$\sin x$	0.9093	$\partial L/\partial s$	$(\partial L/\partial w) \cdot 1$	13.8186
w	$u + s$	6.9093	$\partial L/\partial y$	$(\partial L/\partial u)\, x$	27.6372
L	w^2	47.7384	$\partial L/\partial x$	$(\partial L/\partial u)\, y + (\partial L/\partial s) \cos x$	35.7052

Six nodes evaluated once each in topological order, then six lines backwards — one pass, both derivatives. Forward mode would need two sweeps here, one seeded at x and one at y.

The only bookkeeping in the whole method

x is used twice, once by the product and once by the sine, so

$$\frac{\partial L}{\partial x} = \underbrace{41.4557}_{\text{via } \times} \underbrace{- 5.7505}_{\text{via sin}} = 35.7052.$$

A node's adjoint is the *sum* of the contributions along its outgoing edges. Miss one edge and the gradient is silently wrong, not absent.
Reverse-mode autodiff is a topological sort, a table of local derivatives, and this sum. There is no other idea in it.

Checked against the library, `torch` returns $(35.705220,\ 27.637190)$ — agreeing with the hand derivatives to machine precision. The same habit as Lecture 2's orthogonality test.

2.3 The argument that settles it

Training differentiates a function from 266,610 parameters to *one* scalar loss. Forward mode needs one sweep per input: 266,610 of them. Reverse mode needs one per output: 1.

Reverse mode is not a good choice

It is the *only viable* one, by a factor equal to the parameter count.

And the loss being one number is not a convention. Gradient descent needs a direction of steepest descent, which is only defined for a scalar-valued function; a vector-valued objective has a Jacobian, not a gradient, and no canonical direction to step in. So the shape of the training problem — many parameters, one number — is fixed by the optimisation, and that shape is precisely the one reverse mode is cheap for. The two facts are not independent.

2.4 Measure it rather than trusting it

Small networks, where forward mode can actually be run to completion. Same loss, same point, both modes — and the two columns agree to 2.235×10^{-8}, so this is a measurement of cost, not of quality.

Parameters	Reverse, whole gradient	Forward, whole gradient	Ratio
51	0.24 ms	9.3 ms	39×
99	0.22 ms	16.8 ms	76×
195	0.26 ms	35.5 ms	137×
387	0.21 ms	73.1 ms	341×
771	0.27 ms	151.7 ms	572×

Reverse mode is flat in P; forward mode is linear. At the size we actually train, reverse mode costs 1.33 ms for the whole gradient and forward mode 0.78 ms for *one direction* — so all 266,610 directions would take about 3.5 minutes. That last figure is projected, not run, and saying so is the honest form of the claim.

Lecture 9's run took 20 epochs of 430 batches: 8,600 gradients, in 125 seconds. The same 8,600 in forward mode, at the measured per-pass cost, would be **21 days**. The associativity of matrix multiplication is the only reason this course exists.

So what is backpropagation?

Reverse-mode automatic differentiation, applied to a neural network, with the seed set to 1 at the loss. It is not specific to neural networks, it is not an approximation, and it is not gradient descent — it supplies the gradient that descent then uses. The 1986 paper's contribution was noticing this was cheap enough to be practical, not inventing the chain rule.

What reverse mode costs

Memory. Every Jacobian is evaluated at the point the forward pass reached, so every intermediate activation must be kept until the backward pass consumes it. Forward mode stores nothing; reverse mode stores the whole forward trace, and the trace grows with the batch size and with the depth.
Lecture 12 computes the RAM of a convolutional network and finds that the activations, not the parameters, are what exhausts the device. This is why.

3 What the wrapper costs, measured

The accuracy was not the bug. 88.02% against a 10.0% baseline is a real result: the split was clean, the validation set separate, the test set touched once. The bug is that you cannot do anything else.

Same architecture, optimiser and 20 epochs	Wall clock	Test accuracy
Scikit-Learn, CPU	125 s	88.02%
PyTorch, CPU	14 s	88.84%
PyTorch, MPS	14 s	89.24%

9.1× faster for the same model — and look at rows two and three. The accelerator bought 1.00×, that is, nothing. *All of the speed-up is the framework and none of it is the device.*

The 9.1× came from how the arithmetic is dispatched. Scikit-Learn's MLP is written for clarity and generality, in NumPy, with a Python-level loop over batches; PyTorch fuses the same operations into fewer, larger kernels and keeps the tensors in one layout throughout. Same architecture, same optimiser, same epochs, same arithmetic — and 125 seconds against 14, *both on the CPU*.

3.1 So when does the accelerator pay?

A forward pass is $\phi(\mathbf{XW} + \mathbf{b})$ repeated, and a GPU is a machine for large matrix products and very little else. Ours are not large: 266,610 parameters and a batch of 128 is a matrix product that finishes before the cost of dispatching it has been paid.

Hidden layers	Parameters	CPU	MPS	Ratio
100	79,510	0.13 s	0.45 s	0.28×
300, 100	266,610	0.21 s	0.25 s	0.84×
1000, 1000	1,796,010	0.62 s	0.28 s	2.22×
2000, 2000	5,592,010	1.03 s	0.43 s	2.40×

At our size the ratio is *below* 1: the accelerator is slower. It starts winning at about a million parameters, once each batch carries enough arithmetic to cover the dispatch. Measured on an Apple MPS backend; a Colab T4 has a different crossover, in the same place in the argument.

The honest version of "you need a GPU"

You need one when the network is large enough for the device to matter, and not before. Nothing in this course is — every notebook is sized for Colab's free CPU tier, deliberately, so that every number on these pages is one you can reproduce.
Saying "you need a GPU" without measuring where the crossover falls would be exactly the sort of unmeasured claim this course exists to stop.

4 Tensors, autograd, and the loop

A tensor is an array that also knows two things a NumPy array does not: where it lives — an operation between tensors on different devices is an error, and a loud one — and whether to record.

`requires_grad` does not compute a derivative. It says *record what happens to me*. Every operation on a recording tensor creates a node holding the operation and its inputs; the graph is built during the forward pass, by running it, with no separate compilation step; and it is rebuilt from scratch on every batch, which is why a Python `if` inside `forward` just works. That is the forward sweep from §2, with the graph written down as it goes so the reverse sweep has something to walk — and you can inspect it: `r.grad_fn` and `r.grad_fn.next_functions` are the nodes of the worked example, by name.

Evaluation needs no derivatives, so `torch.no_grad()` stops the recording and saves memory proportional to the batch. Forgetting it is slow and memory-hungry, but it is not a correctness bug. The two that follow are.

```
for epoch in range(EPOCHS):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()          # 1  clears p.grad, which backward() adds to
        out  = model(xb)         # 2  the forward sweep; the graph is recorded
        loss = lossf(out, yb)    # 3  reduces the batch to one scalar
        loss.backward()          # 4  the reverse sweep; fills p.grad
        opt.step()               # 5  updates the parameters from p.grad
```

Five lines. There is no hidden machinery: this is the whole of what `fit()` was doing — and every arrow is a line you wrote, so every arrow is a place you can print, clip, log, branch or stop.

Rebuilt in PyTorch the model scores 89.24% against Scikit-Learn's 88.02% in 14 seconds against 125. The accuracies agree, as they must: same architecture, same optimiser, same learning rate, same epochs, same data. *The rebuild is not an improvement — it is the same model.* What we bought is the loop. Overlaying the two learning curves is the first check after any rewrite; if they had disagreed, one of the two implementations would be wrong.

5 Two bugs that never raise

5.1 The missing `zero_grad()`

`backward()` *accumulates*: `p.grad` is added to, not overwritten. After one call the gradient norm is 1.6183, after two 3.2366, after three 4.8549, and after `zero_grad()` it is 0.0000.

That behaviour is deliberate and useful — it is how you accumulate a large effective batch across several small ones, and how you sum gradients from two loss terms. It is only a bug when you did not ask for it.

What it does to training

At step t the optimiser sees the sum of every gradient computed so far, not the current one. The effective step size grows without bound, and Adam's normalisation hides the divergence for a while. No exception, no warning, no `NaN`.

	Final training loss	Validation accuracy
With `opt.zero_grad()`	0.2574	87.94%
Without it	2.2933	11.20%

76.7 points, on identical seeds and identical everything else.

To catch it: read the loop for the five lines in order — a checklist item, not an insight — and print the gradient norm for the first few steps, since if it climbs monotonically it is accumulating. Watch *where* it sits, too: `zero_grad()` in the epoch loop instead of the batch loop is the same bug, indented two spaces differently, and it survives review.

The general habit

When a framework's default is "accumulate", the code that does *not* accumulate is the code with an extra line in it. Ask what happens if that line is deleted — for every line in the loop.

5.2 The missing `model.eval()`

Some layers behave differently in the two phases. Dropout zeroes a random fraction of units in `train()` and passes everything through in `eval()`; batch normalisation uses this batch's statistics in one and the running averages in the other. *The module defaults to training mode*, so building a model and never calling `eval()` means every evaluation you make is of a randomly crippled network.

Evaluation mode	Test accuracy
`model.eval()`	86.77%
`model.train()`, mean of 10 calls	86.26%
lowest of those 10	86.06%
highest of those 10	86.68%

The tell is the spread, not the mean

0.62 accuracy points across ten *identical* calls. A deterministic function of fixed weights and fixed data returns the same number every time, so if your metric wobbles between calls, ask which layer is still in training mode.

Run any evaluation twice. It costs one line, and it catches this bug, the seeding bugs, and any accidental shuffling of the labels.

Both bugs are one line, neither raises, neither warns, and both produce a plausible number — costing 76.7 points and 0.51 points respectively. One is caught by reading the loop, the other by running it twice, and both survive a review that is looking for exceptions rather than for behaviour.

"Running it twice" needs one caveat

A fixed seed does not make a GPU run bit-reproducible. cuDNN picks convolution algorithms by benchmarking them, and reductions that accumulate with atomics add their terms in whatever order the blocks finish; neither is seeded, so two runs of identical code differ in about the third decimal. `torch.use_deterministic_algorithms(True)` buys determinism back at a cost in speed.

So calibrate what counts as evidence: without those flags a difference in the third decimal means nothing. The two bugs above cost 76.7 and 0.51 points — both far outside it.

6 Feeding the loop, and one more silent error

`Dataset` answers two questions — how many, and give me number i — and `DataLoader` shuffles, batches, and can load in parallel while the device is busy. Indexing a big tensor by hand works while the data fits in memory; it stops working in Lecture 12, and the interface does not change.

The last batch is short

10,000 test images in batches of 384 gives 27 batches, the last containing 16 — so a plain mean of per-batch accuracies weights those 16 exactly as heavily as a full batch.

How the accuracy was aggregated	Value
Counted over the whole set	89.240%
Mean of the per-batch accuracies	89.622%

0.382 points — small *here*, because the last batch is 16 of 384 and the model is no better or worse on it. Change either and it grows: with batch size 3,000 on 10,000 images the short batch carries a quarter of the weight instead of a tenth.

`drop_last=False` is the default and that is the right default. The bug is in how the metric is aggregated, not in the loader.

Weight by the batch size, or count over the set — never take a plain mean of per-batch metrics. Not because the error is always large, but because *you cannot tell from the number whether it was*. That is the identical argument, in the identical shape, as Lecture 2's scale-before-split leak, where the damage was nothing measurable and the rule stayed procedural for the same reason. And it is worse for losses than for accuracy: a per-batch mean of a per-batch mean is two averagings deep.

Subclassing `nn.Module` is what you reach for when `Sequential` runs out — two inputs or two outputs, a skip connection, anything conditional on the data. All three are ordinary Python inside `forward`, because the graph is rebuilt by running the function on every batch. One rule: `super().__init__()` before anything else, or parameter registration silently does not happen.

7 The notebook

What to change

The notebook prints which device it found in its first cell. Leave it on `cpu`: everything here is sized for the free tier, and the crossover measured in §3 shows the accelerator running *this* network slower. Change it only to reproduce that measurement — and then you will have measured your own crossover rather than accepted mine.

8 Exercises

1. The chain rule for a composition is a product of Jacobians. Explain why bracketing it from the output end costs one sweep per output, and state which end is right for training a network. [5]

2. In the worked example $L = (xy + \sin x)^2$, the adjoint of x is a sum of two terms. Say why, and what goes wrong if one is omitted. [4]

3. A run reports 9.1× speed-up moving from Scikit-Learn to PyTorch, and 1.00× moving from CPU to an accelerator. Explain both numbers. [4]

4. A training loop omits `opt.zero_grad()`. State what the optimiser sees at step t, why no exception is raised, and one diagnostic that would reveal it in the first few steps. [5]

5. An evaluation returns 86.26%, 86.68% and 86.06% on three identical calls with fixed weights and fixed data. Name the cause and the one-line fix. [4]

Summary

The chain rule is a product of Jacobians and the product may be bracketed from either end. Forward mode costs one sweep per input; reverse mode costs one sweep per output; training has 266,610 inputs and one scalar output, so reverse mode is the only viable choice — by a factor equal to the parameter count. Measured, forward mode is linear in P and reverse mode flat, and Lecture 9's 8,600 gradients would take 21 days instead of 125 seconds. The price is memory: the forward trace must be kept, which is the bill Lecture 12 pays.

Moving to PyTorch bought 9.1× and the accelerator bought nothing, because at 266,610 parameters the matrix product finishes before its dispatch is paid for — the crossover is near a million. The rebuilt model is the *same* model, and what was bought is the five-line loop.

Then two bugs that raise nothing, warn nothing, and return a plausible number. A missing

`zero_grad()` costs 76.7 accuracy points because `backward()` accumulates by design; a missing `model.eval()` costs 0.51, and its tell is a 0.62-point spread across ten identical calls. One is caught by reading the loop, the other by running it twice.